FEDEX | FDE TRAINING PROGRAM

Group 1: EverMind

Summary for instructors - Phase 1 approval gate, Monday 19 October 2026

Group	Group 1, Netherlands cohort - three members
Type	Greenfield - every component is new, including the knowledge base
Gate 1 plan	src/openspec/changes/add-evermind-pipeline/proposal.md
Date	__ October 2026

What we want to build

Problem. A support knowledge base only answers what somebody thought to write down. Every question it misses is answered by a human once and then forgotten, so the same gap costs the same twenty minutes again next week. The content owner has no signal about what to write next; the customer who asked gets nothing.

Solution. EverMind is a batch job that reads the log of unanswered questions, groups the ones that mean the same thing, and decides for each group: a genuine gap, a rephrasing of something we already have, or a question the knowledge base should not answer. Genuine gaps come back as a drafted answer with its sources attached and a confidence level; everything else comes back with a reason. It never publishes anything itself.

Scope. In: grouping, the three-way decision, finding supporting material, drafting, a queue a reviewer reads, and the demonstration data. Out: publishing, live web search, approving automatically, other languages, a review interface, and answering customers directly.

Why this needs an agent

- **It cannot be one model call.** Checking whether an answer already exists means looking in the knowledge base; drafting a grounded one means finding the source material first. Neither is something a model knows on its own.
- **The work carries between steps.** What was found while researching has to still be there when the answer is checked, or there is no way to verify a citation is real.
- **The path branches.** A duplicate or out-of-scope question stops there. Only genuine gaps go on.
- **It ends in a decision, not a conversation.** Every question leaves with an outcome and a reason attached.

Risks and guards

Risk	Guard
An answer the sources do not actually support. A confident wrong entry is worse than no entry - it gets approved, then served to everyone.	A draft may only use material that was genuinely retrieved, and every claim points at its source. A draft with no source is rejected before a reviewer sees it.
Fifteen rephrasings of one question become fifteen near-identical drafts and the reviewer stops reading.	Questions are grouped before anything is drafted, and a group matching an existing entry comes back as a duplicate rather than written up again.
Answers drafted for questions the knowledge base should not answer - account specifics,	Rejecting is a proper outcome with a stated reason. The scope decision happens first, so a rejected question never reaches

Risk	Guard
complaints, commercial terms.	drafting.
A draft reaches the published knowledge base without anyone approving it.	The system has no way to write to it. Its only output is a queue; publishing is a separate step a person takes.

The demonstration data is ours

There is no external system here and nothing to integrate with. The knowledge base, the log of unanswered questions and the source material are generated by a committed script from a fixed seed, so anyone can reproduce them exactly - and a clean checkout runs with no credentials and no network.

That is a choice, not a shortcut. Because we invented the data we could label it: every logged question carries the outcome the system is supposed to produce. That turns testing from “does this look reasonable” into “did it get the right answer”.

Demonstration data	Contents
The knowledge base	45 published entries across six topics - what we are growing.
The question log	59 questions it could not answer, each labelled with the outcome it should produce.
The source material	15 documents, 31 citable passages - the only evidence a draft may rest on.

Some genuine gaps are deliberately left with no supporting source. Any system can draft when the evidence is in front of it; those cases test whether it declines when it is not.

How we will know it works

The labelled question log gives us assertable tests rather than impressions: the ordinary path, a gap with no supporting source, a duplicate, a question that should be refused, and one containing instructions aimed at the system. We also check that a full run leaves the knowledge base byte-for-byte unchanged. Evidence from those runs backs the Go/No-Go decision on Friday, alongside a written peer review of the code.

Architecture considerations

Answered as business positions for this gate. The technical decisions behind each one are deliberately left open until the plan is approved, and will be recorded as decision records during the build.

Consideration	Our position
Security	No real customer data, no real FedEx content, no credentials - the demonstration data is invented. Customer text is treated as information, never as instruction.
Reliability	A draft that cannot be traced to a source never reaches a reviewer. The job runs overnight rather than while someone waits, so a failed run is just retried.
Performance efficiency	Speed is not the constraint - nothing waits on this. What matters is that a day of questions is processed before the morning.
Cost effectiveness	Grouping first is the main saving: a cluster is handled once instead of a dozen times, and duplicates and refusals drop out before the expensive work.
Operational excellence	Every decision leaves a trail back to the material it rested on, which is what makes approval meaningful rather than ceremonial. Reviewer rejections are signal about where the system is weak.
Deployment	Designed for Google Cloud as a scheduled overnight job writing to an approval queue, deliberately outside any live customer path. Nothing is deployed during the capstone.

Team and roles

The roster lists four members for Group 1. Rafael Del Hoyo is no longer participating, so three people cover four roles. Evaluation is shared rather than dropped.

Role	Owner	Owns
Lead	Yasin Durgun	The plan, presenting at this gate, keeping scope in check, the Go/No-Go call.
Repository owner	Fabian William Bakker	Git and branches, keeping credentials out, the clean-checkout test, the release zip.
Evaluation owner	Siva Prasada Reddy Sogalapalli and Fabian William Bakker	Shared, because we are three. The test set, failure coverage, regression evidence, the peer-review record.
Deck owner	_____	To be agreed Monday morning.

Everyone codes. Siva originated the idea; development is shared across all three.

Plan for the week

Day	Milestone
Monday 19	This approval gate. No agent code is written before it is signed off.
Tuesday 20	Demonstration data committed; the groundwork working end to end on a single question.
Wednesday 21	A full batch running start to finish, with the citation guard in place.
Thursday 22	Test set complete including the failure and adversarial cases; peer review written; deck drafted and rehearsed.
Friday 23	Runnable zip and deck delivered by 09:00. Showcase and Go/No-Go.

What we will hand over

- A runnable repository a teammate can unzip, follow the README, and run without any private files.
- Validated structured output rather than free text.
- A test set covering the ordinary path, a failure, an edge case, a refusal and an adversarial input.
- A written peer review with every Fail resolved (RAPID ITC.009).
- A Go, No-Go or Conditional Go decision backed by cited evidence (RAPID ITC.004).
- A five-minute presentation and deck, delivered by 09:00 on Friday 23 October.

What we are asking for. Approval of the problem and the boundaries, so we can start building Tuesday morning. We have deliberately not fixed the internal design yet - whether this is one agent or several is a decision for the build, and we would rather make it against working code than promise it here. If the problem is redirected, our backup is Customs Document Pre-Clearance Triage: rule-bound rather than research-driven, and a deliberately different shape.